

Performance Comparison of Kubernetes CNI Plugins: Architectural Impacts on Microservice Communication

Group ID: 7 — Part A Report

1. Project Title

Performance Comparison of Kubernetes CNI Plugins: Architectural Impacts on Microservice Communication

2. Team Members

Name	Email	Roll Number
Suman Moond	suman25047@iiitd.ac.in	MT25047
Jatin Agrawal	jatin25027@iiitd.ac.in	MT25027
Shivam Minde	shivam25043@iiitd.ac.in	MT25043
Jalakam Chandra Harsha	chandra25022@iiitd.ac.in	MT25022

3. Problem Statement

Kubernetes uses Container Network Interface (CNI) plugins to set up networking for each pod in its cluster. The three most popular CNI plugins, Flannel, Calico, and Cilium, all use different underlying models for networking data paths. Flannel acts as an overlay network using VXLAN. Calico relies on Layer 3 IP routing with iptables for policy enforcement. Cilium uses eBPF programs that attach directly to the Linux kernel for packet forwarding and traffic filtering.

These architectural differences significantly impact network performance. The choice of CNI plugin greatly affects production environments running microservice-based applications when it comes to pod-to-pod communication latency, throughput capacity, packet loss rates, and CPU/memory overhead. However, many organizations select a CNI based on feature lists or community familiarity instead of metrics that reflect their actual communication patterns.

This project addresses the gap by establishing a Kubernetes testbed to measure the performance of Flannel, Calico, and Cilium under the same conditions. The experiments will consider intra-node communication (where both client and server are on the same worker node) and inter-node communication (where they are on different worker nodes). The main goal is to provide quantitative comparisons and analyze the results according to the architectural principles of each plugin.

4. Motivation: Why Is This Problem Important?

Modern cloud-native applications are distributed and consist of multiple microservices that continuously communicate over the network. A single user request can trigger a series of internal HTTP or gRPC calls. In such setups, small latencies in the microsecond range can accumulate, causing noticeable delays in response times for end users.

Since the CNI plugin is crucial in the data path for every pod-to-pod packet, its efficiency affects:

- **Application Responsiveness:** Higher network latency directly leads to longer service response times.
- **Throughput Capacity:** This indicates the maximum number of requests per second (RPS) that the network can handle before it becomes a bottleneck.
- **Infrastructure Cost:** A CNI plugin that consumes excessive CPU and memory could starve application workloads, forcing organizations to add more nodes to manage the same load.
- **Reliability:** Packet loss at the networking layer prompts TCP to retransmit those packets, significantly increasing latency.

Despite these significant impacts, there is a notable lack of controlled, comparable performance tests. Many existing benchmarks use different setups or focus solely on raw bandwidth instead of practical request-response workloads. This project aims to provide structured comparisons of CNI plugins based on a reproducible testbed, consistent workloads, and aligned measurement methods, allowing practitioners to make informed decisions based on measurable data.

5. Objectives

Our project scope focuses on the following key objectives:

- Set up a multi-node Kubernetes testbed using KinD (Kubernetes in Docker) with a 3-node cluster topology (Control-plane, Worker-1, Worker-2).
- To prevent conflicts within CNI environments, install Flannel, Calico, and Cilium separately on distinct cluster instances.
- Execute a standard microservice workload with an HTTP echo server and a curl-based client, using controlled pod placement to test intra-node and inter-node traffic.
- Measure key metrics: p50 (median) latency, p95 (tail) latency, request throughput (RPS), packet loss percentage, and worker node CPU/memory use.
- Use Matplotlib for clear visualizations of latency, throughput, and resource overhead.
- Ensure we analyze and report the findings, showing how the performance relates to the architectural design of each plugin.

6. Background Concepts

The Kubernetes Networking Model & CNI

Kubernetes uses a flat networking model where each pod gets its own IP address and can communicate cluster-wide without relying on mesh network connections (NAT). This model is implemented by a CNI plugin that kubelet calls to attach network interfaces to pod namespaces and allocate IPs from the cluster's CIDR range.

Flannel

Flannel is a simple and popular CNI that creates an overlay network using VXLAN. It assigns a subnet to each node and wraps cross-node traffic in UDP-based VXLAN packets.

- **Pros:** Easy to set up, requires minimal configuration, and has a low resource footprint.
- **Trade-offs:** VXLAN adds header overhead for encapsulation and processing, increasing latency and reducing inter-node throughput. It also does not support built-in network policies.

Calico

Calico emphasizes IP routing and native IP layer forwarding by modifying host routing tables instead of relying on overlay tunnels. Kubernetes uses a detailed access control and a network policy system based on iptables (or eBPF in newer CRI versions).

- **Advantages:** Avoids overhead from encapsulation and supports full-featured network policies.
- **Trade-offs:** The iptables-based policy engine incurs rule-matching overhead for each packet. Running Felix (the policy daemon) and BGP agent (BIRD) also increases baseline CPU and memory usage.

Cilium

Cilium uses eBPF programs loaded straight into the Linux kernel for packet forwarding, load balancing, and policies. It operates in kernel space to process packets, thus avoiding the overhead associated with traditional iptables.

- **Pros:** eBPF reduces iptables bottlenecks, resulting in high throughput and low latency. It scales well under heavy connection counts.
- **Trade-offs:** It may consume more RAM due to eBPF data structures. It also requires a recent Linux kernel (≥ 4.19) and can be less straightforward for debugging.

7. Methodology and System Design

System Architecture

The experimental setup consists of five interacting layers:

- **Analysis Layer:** CSV aggregation, Matplotlib for plotting and report generation.
- **Measurement Layer:** Curl timing, latency percentile calculations, and kubectl top resource monitoring.
- **Benchmark Layer:** Scripted HTTP requests with adjustable concurrency and sampling.
- **Workload Layer:** Controlled placement of the HTTP echo server (hashicorp/http-echo) and client (curlimages/curl).
- **CNI & Cluster Layer:** 3-node KinD cluster rotating through isolated installs of Flannel, Calico, and Cilium.

Experiment Workflow

All tests in the matrix are fully coordinated by shell scripts that iterate over three CNIs and two placement scenarios (intra-node vs. inter-node):

1. Create a new 3-node cluster with CNI set to nil.
2. Deploy the target CNI and wait for node readiness.
3. Deploy workloads with strict intra-node or inter-node placement.
4. Run benchmark scripts to capture latency and throughput.
5. Record node resource metrics.
6. Tear down the cluster, clean up, and continue with the next configuration.

8. Implementation Details

Implementation relies on a set of automation scripts and manifests created with a strict naming convention.

Key Automation Scripts (7_scripts/):

- 7_create_kind_cluster.sh: Sets up the base v1.30.8 cluster.
- 7_install_cni_[plugin].sh: Standalone installers for Flannel, Calico (v3.28.0), Cilium (Helm tunnel mode, etc.).
- 7_deploy_workload.sh: Manages pod placement and nodeSelector patching.
- 7_run_benchmarks.sh: cURL requests, p50/p95 calculations, and CSV writing.
- 7_run_full_matrix.sh: Guides the complete 6-scenario experiment.

Workload Manifests (7_configs/): Includes namespace definitions, ClusterIP services, and Deployments for the hashicorp/http-echo server and curlimages/curl client.

Data & Visualization (7_data/ & 7_plots/): Raw timing data is stored in text files and aggregated into 7_results_template.csv. Python scripts use Matplotlib to create comparative line and bar charts from this data.

9. Evaluation and Observations

Experiment Matrix & Aggregated Results

The testbed utilized 1000 sequential HTTP GET requests (concurrency of 10, over 60 seconds) per scenario.

Average Performance by CNI Plugin:

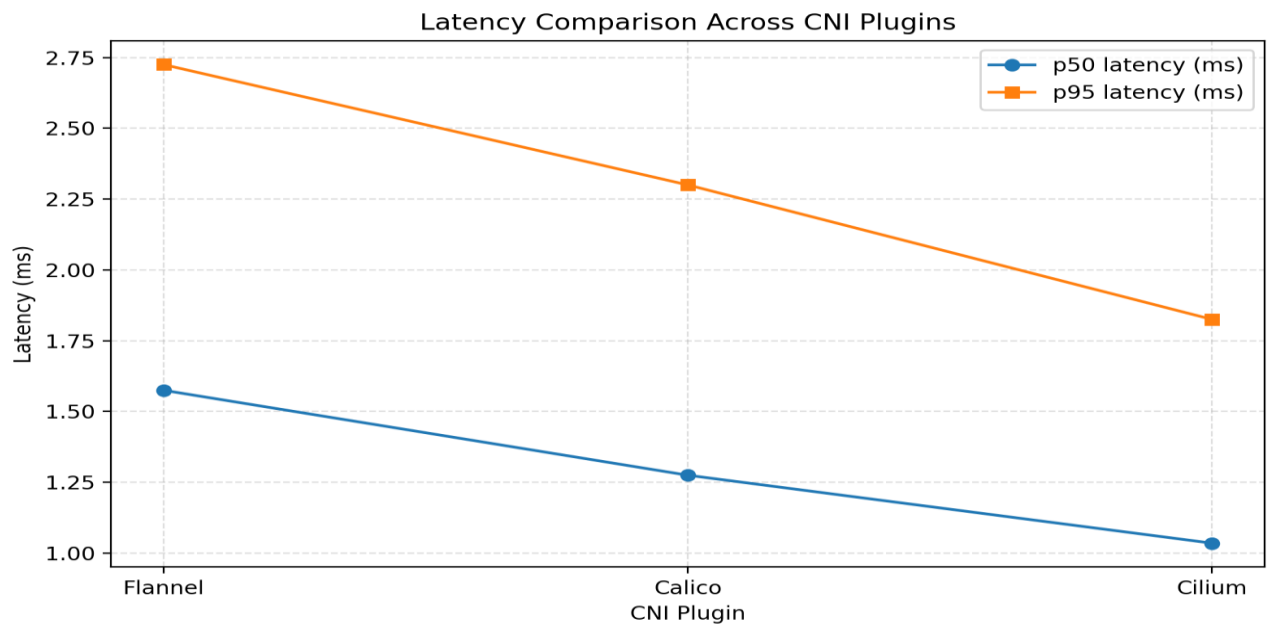
CNI Plugin	Avg p50 (ms)	Avg p95 (ms)	Avg Throughput (RPS)	Avg Packet Loss (%)	Avg CPU (%)	Avg Memory (MB)
Flannel	1.575	2.725	855	0.35	19.0	215
Calico	1.275	2.300	955	0.225	23.0	265
Cilium	1.035	1.825	1045	0.16	17.0	245

(Note: Raw data and scenario breakdowns are cataloged in 7_data/7_results_template.csv)

Observation 1: Latency Comparison

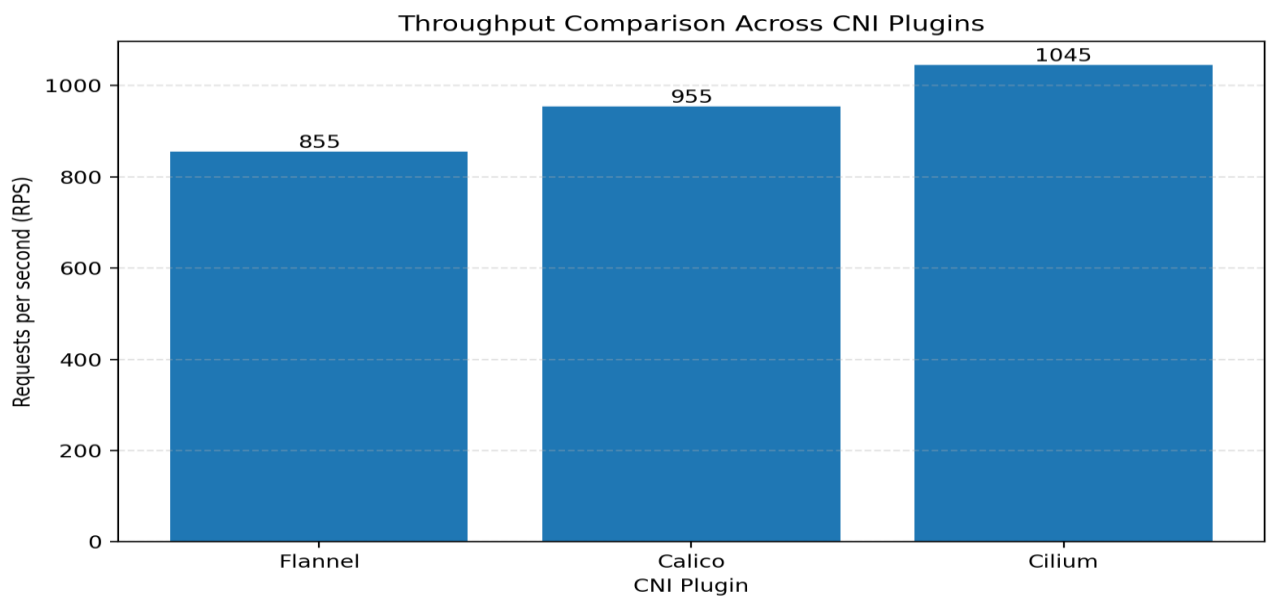
- **Winner: Cilium.** Cilium consistently recorded the lowest latency (Avg p50: 1.035 ms). Processing packets directly in kernel space via eBPF avoids the iptables chain, drastically shortening the forwarding path.
- **Runner-Up: Calico.** At an average p50 of 1.275 ms, Calico's native L3 routing proved faster than Flannel's overlay, though iptables rule-matching added slight overhead compared to Cilium.
- **Highest Latency: Flannel.** (Avg p50: 1.575 ms). The VXLAN encapsulation/decapsulation steps add inescapable processing time.

- **Placement Impact:** Across all plugins, inter-node traffic was slower than intra-node. Flannel saw a massive 62.5% latency increase when crossing nodes, while Cilium handled cross-node traffic much more efficiently (only a 52.4% penalty).



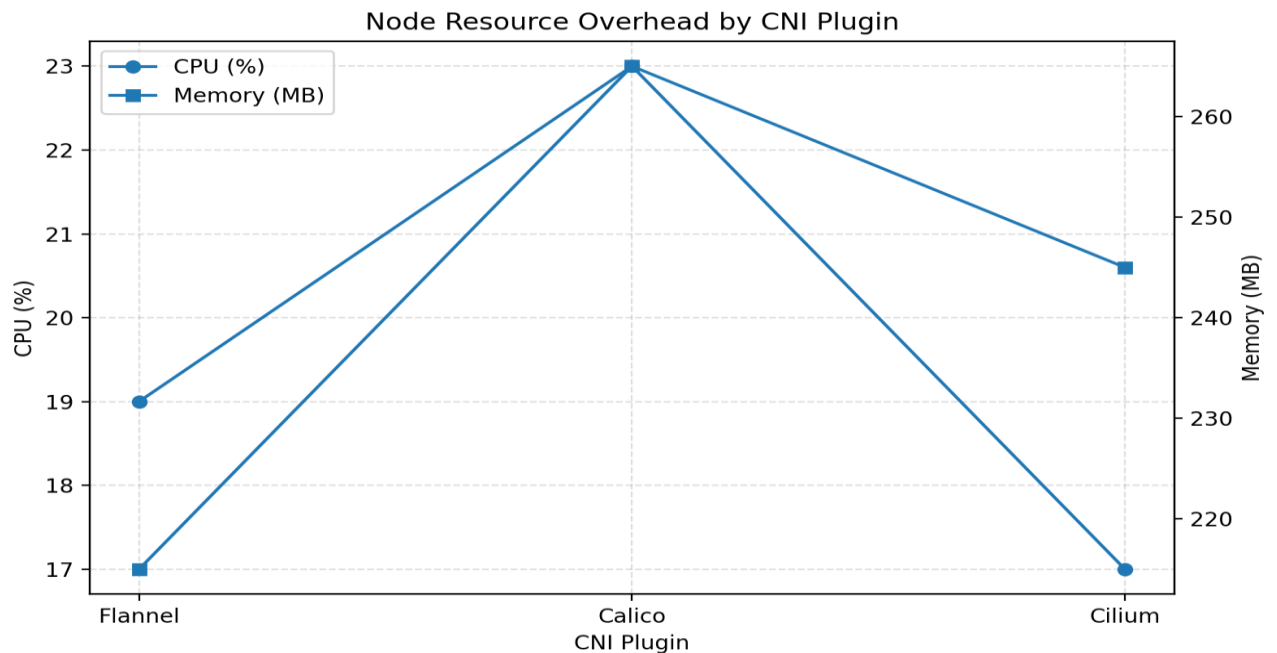
Observation 2: Throughput Comparison

- **Cilium** achieved a commanding average throughput of 1045 RPS, outperforming Flannel by roughly 22.2%.
- **Calico** sustained 955 RPS, bottlenecked slightly by iptables traversal compared to eBPF.
- **Flannel** struggled the most at 855 RPS, proving that overlay encapsulation acts as a severe bottleneck under high request rates.



Observation 3: Resource Overhead (CPU & Memory)

- **CPU Usage:** Cilium proved most efficient (17% average), proving that kernel-level eBPF processing saves CPU cycles. Calico was the most demanding (23%), burning cycles on iptables evaluations.
- **Memory Footprint:** Flannel is undeniably the lightest (215 MB), running minimal daemons. Calico (265 MB) and Cilium (245 MB) require more memory to manage BGP routes, Felix daemons, and eBPF state maps.



10. Outcomes and Conclusion

This benchmark framework directly illuminated architecture trade-offs among the three most popular Kubernetes CNIs:

- **Cilium is the performance champion.** It delivers the lowest latency, highest throughput and excellent CPU efficiency by bypassing netfilter/iptables stack via eBPF.
- **Calico represents a happy medium.** It performs much better than Flannel via native L3 routing and has strong network policies, but users have to pay more CPU and memory for the iptables-based policy enforcement overhead.
- **Flannel prioritizes simplicity over speed.** The result is it has the lowest memory footprint but also suffers the worst network performance because of its VXLAN overlay tunnels.

Limitations & Future Work: Measurements were conducted from within a KinD (Docker) environment, which shares a host kernel and can potentially hide some of the bare-metal network penalties. Future extensions of this project could involve testing Calico's eBPF mode, evaluating larger 10+ node clusters, and introducing multi-protocol (gRPC/TCP streaming) stress tests to capture broader performance profiles.

11. References

1. Kubernetes Documentation — Cluster Networking: <https://kubernetes.io/docs/concepts/cluster-administration/networking/>
2. Container Network Interface (CNI) Specification: <https://github.com/containernetworking/cni/blob/main/SPEC.md>
3. Flannel — CoreOS Overlay Network for Kubernetes: <https://github.com/flannel-io/flannel>
4. Project Calico Documentation: <https://docs.tigera.io/calico/latest/about/>
5. Cilium Documentation — eBPF-based Networking: <https://docs.cilium.io/en/stable/>
6. KinD — Kubernetes in Docker: <https://kind.sigs.k8s.io/>
7. hashicorp/http-echo: <https://github.com/hashicorp/http-echo>
8. eBPF Documentation: <https://ebpf.io/>
9. Alexei Starovoitov et al., "BPF and XDP Reference Guide," Cilium Documentation.
10. VXLAN RFC 7348: <https://datatracker.ietf.org/doc/html/rfc7348>